



618年中复盘会（1/2）—— 25年上半年中大厂高频面试总结

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development T](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题（1/2） --大促搭投实践&高可用性保障](#)

2. 面试常见面试真题解析

2.1 HTTP/HTTPS

2.1.1 HTTP1.0 1.1 2

HTTP1.0

HTTP1.1

HTTP 2

1. 持久连接

HTTP2

1. 1.0规定浏览器和服务器保持短暂的连接。浏览器的每次请求都需要与服务器建立一个TCP连接，服务器处理完后立即断开TCP连接（无连接），服务器不跟踪每个客户端也不记录过去的请求（无状态）

a. 无状态，使用cookie

b. 无法复用连接。每次发送请求的时候，都需要进行一次TCP连接，而TCP的连接释放过程又比较费事的。这种无连接的特性会导致网络的利用率非常低。

c. 队头堵塞(head of line blocking)。由于HTTP/1.0规定下一个请求必须在前一个请求响应到达之前才能发送。假设一个请求响应一直不到达，那么下一个请求就不发送，就会导致阻塞后面的请求。

a. 引入了持久连接，即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive(对于同一个域名，大多数浏览器允许同时建立6个持久连接)

2. 管道机制

a. 即在同一个TCP连接里面，客户端可以同时发送多个请求。

3. 分块传输编码

a. 即服务端每产生一块数据，就发送一块，采用”流模式”而取代”缓存模式”。

4. 新增请求方式

a. PUT:请求服务器存储一个资源;

b. DELETE: 请求服务器删除标识的资源;

c. OPTIONS: 请求查询服务器的性能，或者查询与资源相关的选项和需求;

d. TRACE: 请求服务器回送收到的请求信息，主要用于测试或诊断;

e. CONNECT: 保留将来使用

5. 缺点:

a. 虽然允许复用TCP连接，但是同一个TCP连接里面，所有的数据通信是按次序进行的。服务器只有处理完一个请求，才会接着处理下一个请求。如果前面的处理特别慢，后面就会有許多请求排队等着。这将导致“队头堵塞”

b. 避免方式：一是减少请求数，二是同时多开持久连接

1. 二进制协议

a. HTTP/1.1 版的头信息肯定是文本（ASCII编码），数据体可以是文本，也可以是二进制。HTTP/2 则是一个彻底的二进制协议，头信息和数据体都是二进制，并且统称为”帧”：头信息帧和数据帧。

b. 二进制协议解析起来更高效、“线上”更紧凑，更重要的是错误更少。

2. 完全多路复用

a. HTTP/2 复用TCP连接，在一个连接里，客户端和浏览器都可以同时发送多个请求或回应，而且不用按照顺序一一对应，这样就避免了”队头堵塞”。

3. 报头压缩

a. HTTP 协议是没有状态，导致每次请求都必须附上所有信息。所以，请求的很多头字段都是重复的，比如Cookie，一样的内容每次请求都必须附带，这会浪费很多带宽，也影响速度

b. 对于相同的头部，不必再通过请求发送，只需发送一次;

c. HTTP/2 对这一点做了优化，引入了头信息压缩机制;

d. 一方面，头信息使用gzip或compress压缩后再发送;

e. 另一方面，客户端和服务端同时维护一张头信息表，所有字段都会存入这个表，产生一个索引号，之后就不发送同样字段了，只需发送索引号。

4. 服务器推送

a. HTTP/2 允许服务器未经请求，主动向客户端发送资源;

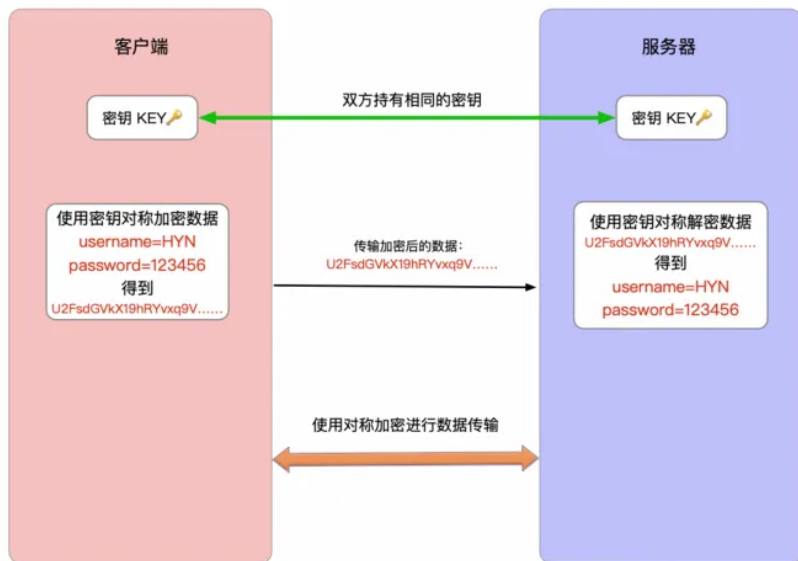
b. 通过推送那些服务器任务客户端将会需要的内容到客户端的缓存中，避免往返的延迟

2.1.2 HTTPS

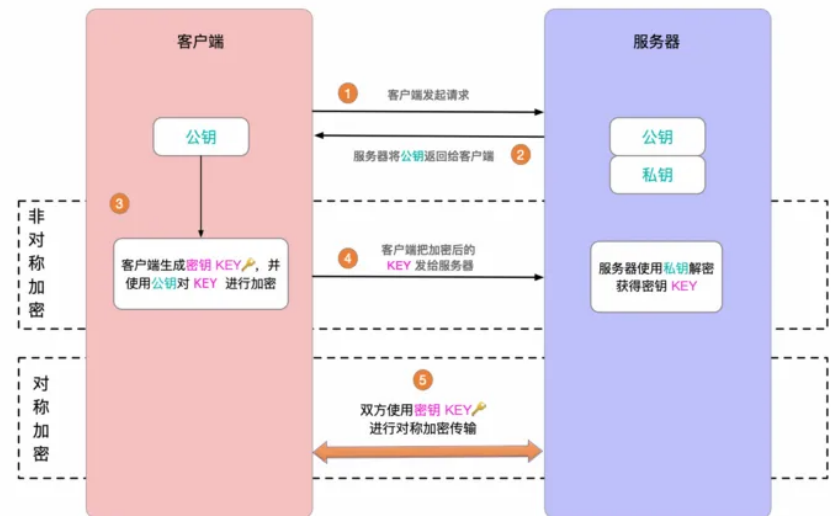
对称加密

非对称加密

对称加密数据传输过程



非对称加密发送 KEY 的过程

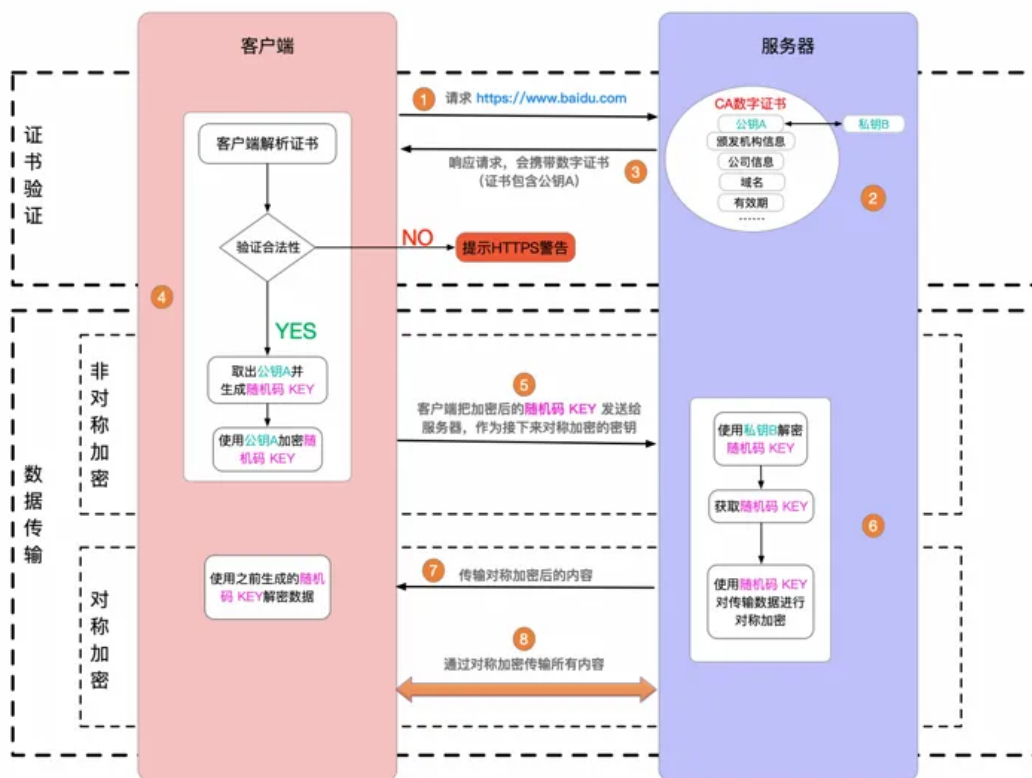


既然双方要使用相同的密钥，那就必然要在传输数据之前先由一方把密钥传给另一方，那么在此过程中密钥就很有可能被截获

HTTPS = HTTP + SSL / TLS。

HTTPS 加密、解密、验证及数据传输过程

by HYN



2.2 跨域

2.2.1 CORS

同源策略是浏览器的一项安全机制，用于限制一个源（origin）的文档或脚本与另一个源的资源进行交互的能力。同源策略的核心规则是：只有当两个URL的协议（protocol）、**域名（host）和端口（port）**都相同时，它们才被认为是同源的。

CORS 的工作原理

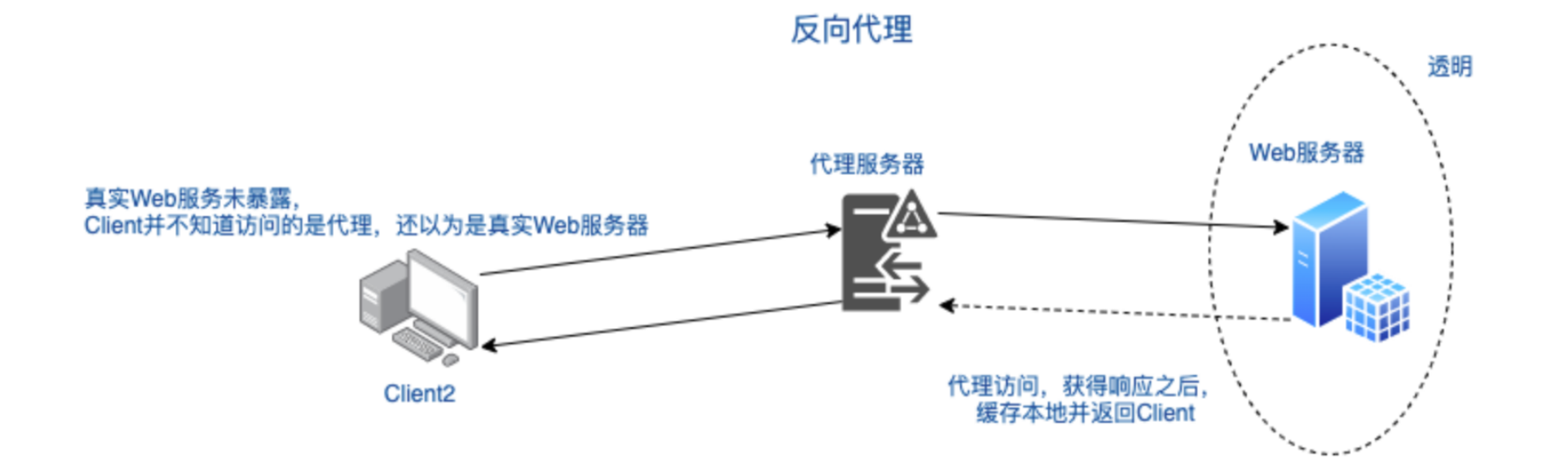
当浏览器发起跨域请求时，它会先发送一个带有 Origin 头的预检请求（preflight request，使用 OPTIONS 方法）。服务器根据请求的 Origin 头决定是否允许该请求，并在响应中返回相应的 CORS 头。

CORS失败原因

a. 预检请求失败：

- 原因：服务器未正确配置 CORS 头。
- 解决方案：确保服务器返回正确的 CORS 头。

b. **CORS 头缺失:**



- a. **负载均衡**: 反向代理可以根据后端服务器的负载情况, 将请求分发到不同的服务器上, 实现负载均衡, 提高系统的整体性能。
- b. **安全性增强**: 反向代理可以隐藏后端服务器的真实地址和端口, 防止直接攻击 (如DoS/DDoS)。同时, 还可以实现SSL加密、访问控制等安全功能。
- c. **缓存优化**: 反向代理可以缓存静态资源, 减少后端服务器的负载, 提高响应速度。

Code block

```
1  server {
2    listen 80;
3    server_name api.your-domain.com;
4    location / {
5      proxy_pass http://target-server:3000;
6      proxy_set_header Host $host;
7      proxy_set_header X-Real-IP $remote_addr;
8      add_header 'Access-Control-Allow-Origin' 'https://your-domain.com' always;
9    }
10 }
```

2.2.3 本地代理

Code block

```
1  // vite.config.js
2  export default defineConfig({
3    server: {
4      proxy: {
5        '/api': {
6          target: 'http://backend:3000',
7          changeOrigin: true,
8          rewrite: path => path.replace(/^\/api/, '')
9        }
10     }
11   })
12 })
```

2.2.4 JSONP(GET)

Code block

```
1  function jsonp(url, callback) {
2    const script = document.createElement('script');
```

```
3     script.src = `${url}?callback=${callback.name}`;
4     document.body.appendChild(script);
5 }
6
7 function handleResponse(data) {
8     console.log('Received:', data);
9 }
```

2.2.5 PostMessage

Code block

```
1 // 父窗口
2 iframe.contentWindow.postMessage('hello', 'https://child-domain.com');
3
4 // 子窗口监听
5 window.addEventListener('message', event => {
6     if (event.origin !== 'https://parent-domain.com') return;
7     console.log('Received:', event.data);
8 });
```

2.2.6 WebSocket

Code block

```
1 const socket = new WebSocket('wss://your-websocket-server');
2 socket.onopen = () => {
3     socket.send(JSON.stringify({ type: 'ping' }));
4 };
```

2.3 攻击

2.3.1 SQL注入

攻击者通过在输入字段中注入恶意 SQL 代码，从而篡改数据库查询。

防御措施：

- **使用参数化查询：**通过预编译的 SQL 语句和参数化查询，避免直接拼接 SQL。
- **ORM 框架：**使用 ORM 框架（如 Hibernate、Entity Framework）来处理数据库操作。
- **输入验证：**对用户输入进行严格的验证，确保输入符合预期格式。
- **Web 应用防火墙（WAF）：**部署 WAF 来自动检测和过滤恶意 SQL 注入请求。

2.3.2 XSS（Cross-site scripting）

攻击通过在网页中注入恶意脚本，当这些脚本在用户的浏览器中执行时，可能会窃取用户数据或执行恶意操作。

- 存储型 XSS
 - 注入型脚本永久存储在目标服务器上。当浏览器请求数据时，脚本从服务器上传并执行。
- 反射型 XSS
 - 当用户点击一个恶意链接，或者提交一个表单，或者进入一个恶意网站时，注入脚本进入被攻击者的网站。Web 服务器将注入脚本，比如一个错误信息，搜索结果等 返回到用户的浏览器上。由于浏览器认为这个响应来自"可信任"的服务器，所以会执行这段脚本。
- 基于 DOM 的 XSS

- 通过修改原始的客户端代码，受害者浏览器的 DOM 环境改变，导致有效载荷的执行。也就是说，页面本身并没有变化，但由于 DOM 环境被恶意修改，有客户端代码被包含进了页面，并且意外执行

防御措施：

- **输入验证与净化**：对用户输入进行严格的验证和净化，移除潜在的危险字符。
- **输出编码**：在将数据插入 HTML、JavaScript、CSS 或 URL 中时，进行适当的编码。
- **设置 HttpOnly Cookie**：将重要 Cookie 设置为 HttpOnly，防止 JavaScript 访问。

2.3.3 CSRF (Cross-site request forgery) 跨站请求伪造

攻击者诱导受害者进入第三方网站，在第三方网站中，向被攻击网站发送跨站请求。利用受害者在被攻击网站已经获取的注册凭证，绕过后台的用户验证，达到冒充用户对被攻击的网站执行某项操作的目的。

- 一个典型的CSRF攻击有着如下的流程：
 - 受害者登录[a.com](#)，并保留了登录凭证（Cookie）。
 - 攻击者引诱受害者访问了[b.com](#)。
 - [b.com](#) 向 [a.com](#) 发送了一个请求：[a.com/act=xx](#)。浏览器会默认携带[a.com](#)的Cookie。
 - [a.com](#)接收到请求后，对请求进行验证，并确认是受害者的凭证，误以为是受害者自己发送的请求。
 - [a.com](#)以受害者的名义执行了act=xx。
 - 攻击完成，攻击者在受害者不知情的情况下，冒充受害者，让[a.com](#)执行了自己定义的操作。
- 处理：cookie samesite

2.4 Cookie

2.4.1 Secure 和 HttpOnly 属性

Secure 属性和HttpOnly 属性

- Secure：表示只应通过被 HTTPS 协议加密过的请求发送给服务端；
- HttpOnly：JavaScript Document.cookie API 无法访问带有 HttpOnly 属性的 cookie；此类 Cookie 仅作用于服务器。例如，持久化服务器端会话的 Cookie 不需要对 JavaScript 可用，而应具有 HttpOnly 属性。

2.4.2 SameSite

SameSite 可以有下面三种值：

1. None：浏览器会在同站请求、跨站请求下继续发送 cookies，不区分大小写；
2. Strict：浏览器将只在访问相同站点时发送 cookie；
3. Lax：与 Strict 类似，但用户从外部站点导航至 URL 时（例如通过链接）除外。在新版本浏览器中，为默认选项，Same-site cookies 将会为一些跨站子请求保留，如图片加载或者 frames 的调用，但只有当用户从外部站点导航到 URL 时才会发送。如 link 链接；

大多数主流浏览器基本上已经将 SameSite 的默认值迁移至 Lax。如果想要指定 Cookies 在同站、跨站请求都被发送，现在需要明确指定 SameSite 为 None；

2.4.3 安全性

1. 使用 HttpOnly 属性可防止通过 JavaScript 访问 cookie 值；
2. 用于敏感信息（例如指示身份验证）的 Cookie 的生存期应较短，并且 SameSite 属性设置为Strict 或 Lax

2.5 缓存

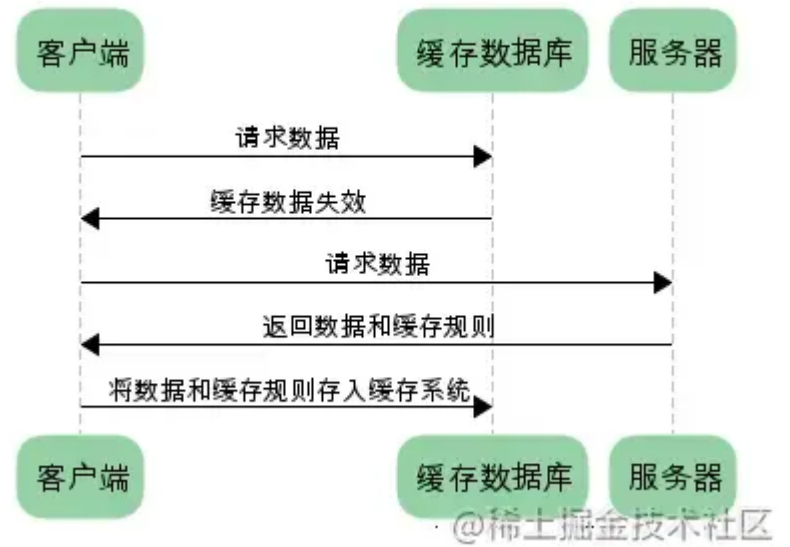
2.5.1 强制缓存

强制缓存不需要与服务器发生交互。

强制缓存规则下，缓存命中



强制缓存规则下，缓存未命中



Expires

HTTP 1.0 的字段，表示缓存到期时间，是一个绝对的时间 (当前时间+缓存时间)。在响应消息头中，设置这个字段之后，就可以告诉浏览器，在未过期之前不需要再次请求。

缺点：若修改电脑的本地时间，会导致浏览器判断缓存失效 这里修重新修改缓存

Cache-control

HTTP/1.1

1. no-cache：告诉浏览器忽略资源的缓存副本，强制每次请求直接发送给服务器，拉取资源，但不是“不缓存”，相当于需要使用协商缓存，禁止使用强制缓存；
2. no-store：强制缓存在任何情况下都不要保留任何副本，相当于不使用强制缓存和协商缓存；
3. public 任何路径的缓存者（本地缓存、代理服务器），可以无条件的缓存改资源，不设置默认为public；
4. private 只针对单个用户或者实体（不同用户、窗口）缓存资源；

2.5.2 对比缓存

last-modified

1. 服务器在响应头中设置last-modified字段返回给客户端，告诉客户端资源最后一次修改的时间；
 - a. `Last-Modified: Sat, 30 Mar 2029 05:46:11 GMT`
2. 浏览器在这个值和内容记录在浏览器的缓存数据库中；
3. 下次请求相同资源，浏览器将在请求头中设置if-modified-since的值（这个值就是第一步响应头中的Last-Modified的值）传给服务器；
4. 服务器收到请求头的if-modified-since的值与last-modified的值比较，如果相等，表示未进行修改，则返回状态码为304；如果不相等，则修改了，返回状态码为200，并返回数据；

Etag

tag是根据文件内容，算出一个唯一的值。服务器存储着文件的Etag 字段。

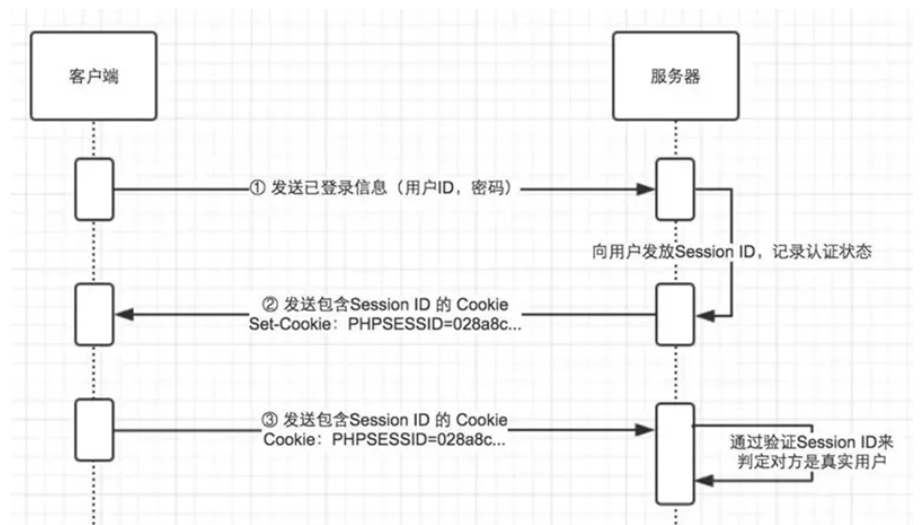
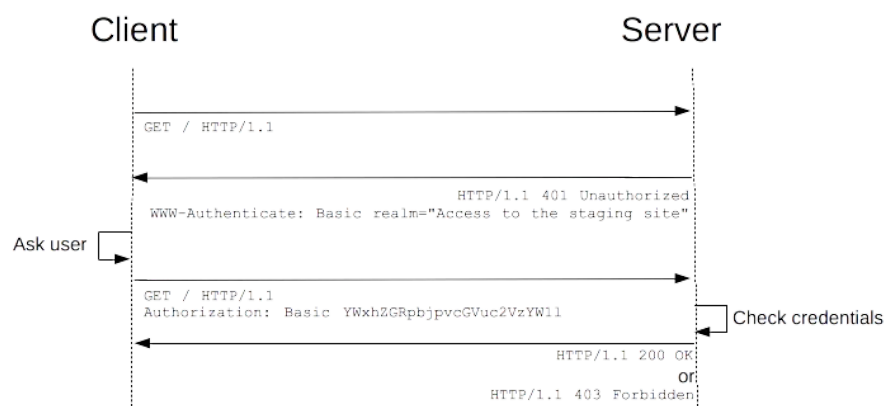
If-None-Match。

服务器同样进行比较，命中返回 304, 不命中返回新资源和 200。 Etag 的优先级高于 Last-Modified

2.6 鉴权

HTTP Basic Authentication

session-cookie



Token JWT (JSON Web Token)

AI智能助手 使用 Authorization

1. 客户端使用用户名跟密码请求登录；
2. 服务端收到请求，去验证用户名与密码；
3. 验证成功后，服务端会签发一个 Token，再把这个 Token 发送给客户端；
4. 客户端收到 Token 以后可以把它存储起来，比如放在 Cookie 里或者 Local Storage 里；
5. 客户端每次向服务端请求资源的时候需要带着服务端签发的 Token；
6. 服务端收到请求，然后去验证客户端请求里面带着的 Token（request 头部添加 Authorization），如果验证成功，就向客户端返回请求的数据，如果不成功返回 401 错误码，鉴权失败

跟 cookie-session 区别

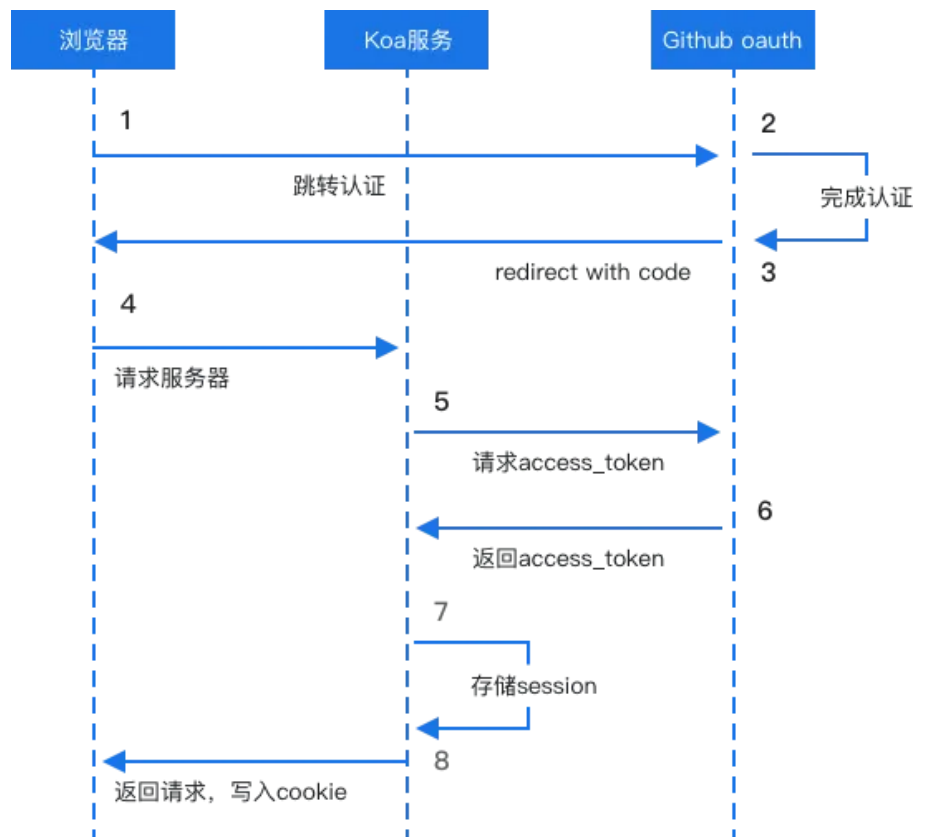
cookie-session

1. 认证方式局限于在浏览器中使用，cookie 是浏览器端的机制，如果在 app 端就无法使用 cookie；
2. 为了满足全局一致性，我们最好把 session 存储在 redis 中做持久化，而在分布式环境下，我们可能需要在每个服务器上都备份，占用了大量的存储空间；
3. 在不是 Https 协议下使用 cookie，容易受到 CSRF 跨站点请求伪造攻击。

token 的缺点：

1. 加密解密消耗使得 token 认证比 session-cookie 更消耗性能；
2. token 比 sid 大，更占带宽；

OAuth



Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + ".",
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

3. 前端模块化加载原理实践

3.1 Node端 CommonJS

`node` 中是自带模块化机制的，每个文件就是一个单独的模块，并且它遵循的是 `CommonJS` 规范，也就是使用 `require` 的方式导入模块，通过 `module.export` 的方式导出模块。

`node` 模块的运行机制也很简单，其实就是在每一个模块外层包裹了一层函数，有了函数的包裹就可以实现代码间的作用域隔离。

新建一个 `js` 文件，在第一行打印一个并不存在的变量，比如打印 `window`，在 `node` 中是没有 `window` 的。

Code block

```
1 console.log(window);
```

通过 `node` 执行该文件，会发现报错信息如下。(请使用系统默认 `cmd` 执行命令)。

Code block

```
1 (function (exports, require, module, __filename, __dirname) { console.log(window);
2 ReferenceError: window is not defined
3   at Object.<anonymous> (/Users/choice/Desktop/node/main.js:1:75)
4   at Module._compile (internal/modules/cjs/loader.js:689:30)
5   at Object.Module._extensions..js (internal/modules/cjs/loader.js:700:10)
6   at Module.load (internal/modules/cjs/loader.js:599:32)
7   at tryModuleLoad (internal/modules/cjs/loader.js:538:12)
8   at Function.Module._load (internal/modules/cjs/loader.js:530:3)
9   at Function.Module.runMain (internal/modules/cjs/loader.js:742:12)
10  at startup (internal/bootstrap/node.js:279:19)
11  at bootstrapNodeJSCore (internal/bootstrap/node.js:752:3)
```

可以看到报错的顶层有一个自执行的函数，，函数中包含 `exports`，`require`，`module`，`__filename`，`__dirname` 这些常用的全局变量。

在模块系统中，每个文件就是一个模块，每个模块外面会自动套一个函数，并且定义了导出方式 `module.exports` 或者 `exports`，同时也定义了导入方式 `require`。

Code block

```
1 let moduleA = (function() {
2   module.exports = Promise;
3   return module.exports;
4 })();
```

3.1.1 require加载模块

`require` 依赖 `node` 中的 `fs` 模块来加载模块文件，`fs.readFile` 读取到的是一个字符串。

在 `javascript` 中可以通过 `eval` 或者 `new Function` 的方式来将一个字符串转换成 `js` 代码来运行。

3.1.1.1 Eval

Code block

```
1  const name = 'chenghuai';
2  const str = 'const a = 123; console.log(name)';
3  eval(str); // chenghuai;
```

3.1.1.2 new Function

`new Function` 接收的是一个要执行的字符串，返回的是一个新的函数，调用这个新的函数字符串就会执行了。如果这个函数需要传递参数，可以在 `new Function` 的时候依次传入参数，最后传入的是要执行的字符串。比如这里传入参数 `b`，要执行的字符串 `str`。

Code block

```
1  const b = 3;
2  const str = 'let a = 1; return a + b';
3  const fun = new Function('b', str);
4  console.log(fun(b, str)); // 4
```

可以看到 `eval` 和 `Function` 实例化都可以用来执行 `javascript` 字符串，似乎他们都可以来实现 `require` 模块加载。不过在 `node` 中并没有选用他们来实现模块化，原因也很简单因为他们都有一个致命的问题，就是都容易被不属于他们的变量所影响。

如下 `str` 字符串中并没有定义 `a`，但是确可以使用上面定义的 `a` 变量，这显然是不对的，在模块化机制中，`str` 字符串应该具有自身独立的运行空间，自身不存在的变量是不可以直接使用的。

Code block

```
1  const a = 1;
2
3  const str = 'console.log(a)';
4
5  eval(str);
6
7  const func = new Function(str);
8  func();
```

`node` 存在一个 `vm` 虚拟环境的概念，用来运行额外的 `js` 文件，他可以保证 `javascript` 执行的独立性，不会被外部所影响。

3.1.1.3 vm 内置模块

虽然外部定义了 `hello`，但是 `str` 是一个独立的模块，并不在 `hello` 变量，所以会直接报错。

Code block

```
1  // 引入vm模块， 不需要安装，node 自建模块
2  const vm = require('vm');
3  const hello = 'yd';
4  const str = 'console.log(hello)';
5  vm.runInThisContext(str); // 报错
```

所以 `node` 执行 `javascript` 模块时可以采用 `vm` 来实现。就可以保证模块的独立性了。

3.1.2 require实现

3.1.2.1 path模块

用于处理文件路径。

- basename: 基础路径, 有文件路径就不是基础路径, 基础路径是 `1.js`
- extname: 获取扩展名
- dirname: 父级路劲
- join: 拼接路径
- resolve: 当前文件夹的绝对路径, 注意使用的时候不要在结尾添加 `/`
- __dirname: 当前文件所在文件夹的路径
- __filename: 当前文件的绝对路径

Code block

```
1  const path = require('path', 's');
2  console.log(path.basename('1.js'));
3  console.log(path.extname('2.txt'));
4  console.log(path.dirname('2.txt'));
5  console.log(path.join('a/b/c', 'd/e/f')); // a/b/c/d/e/
6  console.log(path.resolve('2.txt'));
```

3.1.2.2 fs模块

用于操作文件或者文件夹, 比如文件的读写, 新增, 删除等。常用方法有 `readFile` 和 `readFileSync`, 分别是异步读取文件和同步读取文件。

Code block

```
1  const fs = require('fs');
2  const buffer = fs.readFileSync('./name.txt', 'utf8'); // 如果不传入编码, 出来的是二进制
3  console.log(buffer);
```

`fs.access`: 判断是否存在, `node10` 提供的, `exists` 方法已经被废弃, 原因是不符合 `node` 规范, 所以改为采用 `access` 来判断文件是否存在。

Code block

```
1  try {
2      fs.accessSync('./name.txt');
3  } catch(e) {
4      // 文件不存在
5  }
```

3.1.3 手动实现require模块加载器

首先导入依赖的模块 `path`, `fs`, `vm`, 并且创建一个 `Require` 函数, 这个函数接收一个 `modulePath` 参数, 表示要导入的文件路径。

Code block

```
1  // 导入依赖
2  const path = require('path'); // 路径操作
3  const fs = require('fs'); // 文件读取
4  const vm = require('vm'); // 文件执行
5
6  // 定义导入类, 参数为模块路径
7  function Require(modulePath) {
```

```
8     ...
9 }
```

在 `Require` 中获取到模块的绝对路径，方便使用 `fs` 加载模块，这里读取模块内容使用 `new Module` 来抽象，使用 `tryModuleLoad` 来加载模块内容，`Module` 和 `tryModuleLoad` 稍后实现，`Require` 的返回值应该是模块的内容，也就是 `module.exports`。

Code block

```
1 // 定义导入类，参数为模块路径
2 function Require(modulePath) {
3     // 获取当前要加载的绝对路径
4     let absPathname = path.resolve(__dirname, modulePath);
5     // 创建模块，新建Module实例
6     const module = new Module(absPathname);
7     // 加载当前模块
8     tryModuleLoad(module);
9     // 返回exports对象
10    return module.exports;
11 }
12
```

`Module` 的实现很简单，就是给模块创建一个 `exports` 对象，`tryModuleLoad` 执行的时候将内容加入到 `exports` 中，`id` 就是模块的绝对路径。

Code block

```
1 // 定义模块，添加文件id标识和exports属性
2 function Module(id) {
3     this.id = id;
4     // 读取到的文件内容会放在exports中
5     this.exports = {};
6 }
```

之前说过 `node` 模块是运行在一个函数中，这里给 `Module` 挂载静态属性 `wrapper`，里面定义一下这个函数的字符串，`wrapper` 是一个数组，数组的第一个元素就是函数的参数部分，其中有 `exports`，`module`，`Require`，`__dirname`，`__filename`，都是模块中常用的全局变量。注意这里传入的 `Require` 参数是定义的 `Require`。

第二个参数就是函数的结束部分。两部分都是字符串，使用的时候将他们包裹在模块的字符串外部就可以了。

Code block

```
1 Module.wrapper = [
2     "(function(exports, module, Require, __dirname, __filename) {",
3     "})"
4 ]
```

`_extensions` 用于针对不同的模块扩展名使用不同的加载方式，比如 `JSON` 和 `javascript` 加载方式肯定是不同的。`JSON` 使用 `JSON.parse` 来运行。

`javascript` 使用 `vm.runInThisContext` 来运行，可以看到 `fs.readFileSync` 传入的是 `module.id` 也就是 `Module` 定义时候 `id` 存储的是模块的绝对路径，读取到的 `content` 是一个字符串，使用 `Module.wrapper` 来包裹一下就相当于在这个模块外部又包裹了一个函数，也就实现了私有作用域。

使用 `call` 来执行 `fn` 函数，第一个参数改变运行的 `this` 传入 `module.exports`，后面的参数就是函数外面包裹参数 `exports`，`module`，`Require`，`__dirname`，`__filename`。

Code block

```

1  Module._extensions = {
2    '.js'(module) {
3      const content = fs.readFileSync(module.id, 'utf8');
4      const fnStr = Module.wrapper[0] + content + Module.wrapper[1];
5      const fn = vm.runInThisContext(fnStr);
6      fn.call(module.exports, module.exports, module, Require, _filename, _dirname);
7    },
8    '.json'(module) {
9      const json = fs.readFileSync(module.id, 'utf8');
10     module.exports = JSON.parse(json); // 把文件的结果放在exports属性上
11   }
12 }

```

`tryModuleLoad` 函数接收的是模块对象，通过 `path.extname` 来获取模块的后缀名，然后使用 `Module._extensions` 来加载模块。

Code block

```

1  // 定义模块加载方法
2  function tryModuleLoad(module) {
3    // 获取扩展名
4    const extension = path.extname(module.id);
5    // 通过后缀加载当前模块
6    Module._extensions[extension](module);
7  }

```

至此 `Require` 加载机制基本就写完了。`Require` 加载模块的时候传入模块名称，在 `Require` 方法中使用 `path.resolve(__dirname, modulePath)` 获取到文件的绝对路径。然后通过 `new Module` 实例化的方式创建 `module` 对象，将模块的绝对路径存储在 `module` 的 `id` 属性中，在 `module` 中创建 `exports` 属性为一个 `json` 对象。

使用 `tryModuleLoad` 方法去加载模块，`tryModuleLoad` 中使用 `path.extname` 获取到文件的扩展名，然后根据扩展名来执行对应的模块加载机制。

最终将加载到的模块挂载 `module.exports` 中。`tryModuleLoad` 执行完毕之后 `module.exports` 已经存在了，直接返回就可以了。

Code block

```

1  // 导入依赖
2  const path = require('path'); // 路径操作
3  const fs = require('fs'); // 文件读取
4  const vm = require('vm'); // 文件执行
5
6  // 定义导入类，参数为模块路径
7  function Require(modulePath) {
8    // 获取当前要加载的绝对路径
9    let absPathname = path.resolve(__dirname, modulePath);
10    // 创建模块，新建Module实例
11    const module = new Module(absPathname);
12    // 加载当前模块
13    tryModuleLoad(module);
14    // 返回exports对象
15    return module.exports;
16  }
17  // 定义模块，添加文件id标识和exports属性
18  function Module(id) {
19    this.id = id;
20    // 读取到的文件内容会放在exports中
21    this.exports = {};
22  }

```

```

23 // 定义包裹模块内容的函数
24 Module.wrapper = [
25     "(function(exports, module, Require, __dirname, __filename) {" ,
26     "})"
27 ]
28 // 定义扩展名，不同的扩展名，加载方式不同，实现js和json
29 Module._extensions = {
30     '.js'(module) {
31         const content = fs.readFileSync(module.id, 'utf8');
32         const fnStr = Module.wrapper[0] + content + Module.wrapper[1];
33         const fn = vm.runInThisContext(fnStr);
34         fn.call(module.exports, module.exports, module, Require, _filename, _dirname);
35     },
36     '.json'(module) {
37         const json = fs.readFileSync(module.id, 'utf8');
38         module.exports = JSON.parse(json); // 把文件的结果放在exports属性上
39     }
40 }
41 // 定义模块加载方法
42 function tryModuleLoad(module) {
43     // 获取扩展名
44     const extension = path.extname(module.id);
45     // 通过后缀加载当前模块
46     Module._extensions[extension](module);
47 }

```

3.1.4 添加缓存

添加缓存也比较简单，就是文件加载的时候将文件放入缓存在，再去加载模块时先看缓存中是否存在，如果存在直接使用，如果不存在再去重新加载，加载之后再放入缓存。

Code block

```

1 // 定义导入类，参数为模块路径
2 function Require(modulePath) {
3     // 获取当前要加载的绝对路径
4     let absPathname = path.resolve(__dirname, modulePath);
5     // 从缓存中读取，如果存在，直接返回结果
6     if (Module._cache[absPathname]) {
7         return Module._cache[absPathname].exports;
8     }
9     // 创建模块，新建Module实例
10    const module = new Module(absPathname);
11    // 添加缓存
12    Module._cache[absPathname] = module;
13    // 加载当前模块
14    tryModuleLoad(module);
15    // 返回exports对象
16    return module.exports;
17 }

```

3.1.5 省略模块后缀名

自动给模块添加后缀名，实现省略后缀名加载模块，其实也就是如果文件没有后缀名的时候遍历一下所有的后缀名看一下文件是否存在。

Code block

```

1 // 定义导入类，参数为模块路径
2 function Require(modulePath) {

```

```

3      // 获取当前要加载的绝对路径
4      let absPathname = path.resolve(__dirname, modulePath);
5      // 获取所有后缀名
6      const extNames = Object.keys(Module._extensions);
7      let index = 0;
8      // 存储原始文件路径
9      const oldPath = absPathname;
10     function findExt(absPathname) {
11         if (index === extNames.length) {
12             return throw new Error('文件不存在');
13         }
14         try {
15             fs.accessSync(absPathname);
16             return absPathname;
17         } catch(e) {
18             const ext = extNames[index++];
19             findExt(oldPath + ext);
20         }
21     }
22     // 递归追加后缀名，判断文件是否存在
23     absPathname = findExt(absPathname);
24     // 从缓存中读取，如果存在，直接返回结果
25     if (Module._cache[absPathname]) {
26         return Module._cache[absPathname].exports;
27     }
28     // 创建模块，新建Module实例
29     const module = new Module(absPathname);
30     // 添加缓存
31     Module._cache[absPathname] = module;
32     // 加载当前模块
33     tryModuleLoad(module);
34     // 返回exports对象
35     return module.exports;
36 }
37

```

3.1.6 整体链路

1. 导入相关模块，创建一个Require方法。
2. 抽离通过Module._load方法，用于加载模块。
3. Module.resolveFilename 根据相对路径，转换成绝对路径。
4. 缓存模块 Module._cache，同一个模块不要重复加载，提升性能。
5. 创建模块 id: 保存的内容是 exports = {}相当于this。
6. 利用tryModuleLoad(module, filename) 尝试加载模块。
7. Module._extensions使用读取文件。
8. Module.wrap: 把读取到的js包裹一个函数。
9. 将拿到的字符串使用runInThisContext运行字符串。
10. 让字符串执行并将this改编成exports。

3.2 浏览器端 ESM

详情可以参考：<https://hacks.mozilla.org/2018/03/es-modules-a-cartoon-deep-dive/>

3.2.1 ES Modules 流程解析

当前，在浏览器中通过 `<script type="module">` 已原生支持 ESM。以vite创建的Vue3项目为例：

Code block

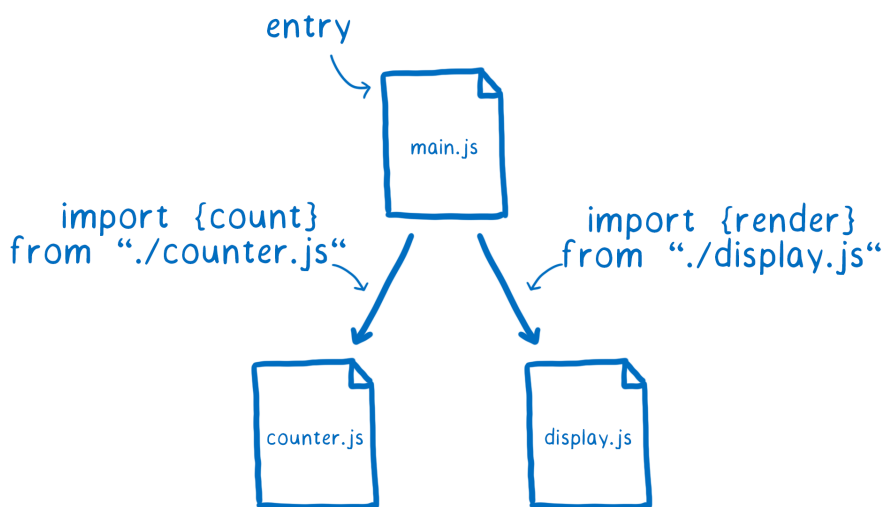
```
1  // index.html文件
2  <!DOCTYPE html>
3  <html lang="en">
4    <head>
5      <meta charset="UTF-8" />
6      <link rel="icon" href="favicon.png" />
7      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
8      <title>xxxx</title>
9    </head>
10   <body>
11     <div id="app"></div>
12     <script type="module" src="/src/main.ts"></script>
13   </body>
14 </html>
```

其中 `/src/main.ts` 是Vue3的入口文件。

注意：浏览器中 `ES Module` 是异步加载，不会堵塞浏览器，即等到整个页面渲染完，再执行模块脚本。如果网页有多个 `ESM`，它们会按照在页面出现的顺序依次执行。

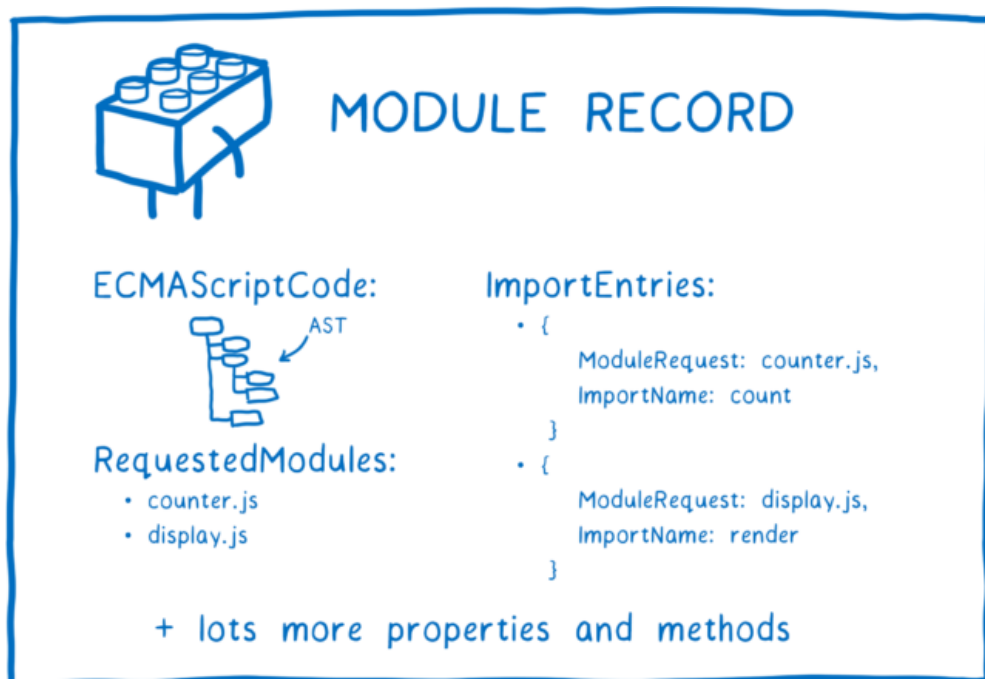
1. 创建AST

当使用 `ES Modules` 进行开发时，实际上以入口节点为根节点（如main.js）创建出一张依赖关系图。不同依赖项之间通过 `export\import` 语句来进行关联。



2. 生成 Module Record

浏览器无法直接使用文件本身，但是浏览器会解析这些文件，根据 `import/export` 语句构成模块记录(`Module Record`)，每个模块文件对应生成一个 `Module Record`，记录当前模块的信息：



3. 转化 Module Instance

模块记录转化为模块实例，浏览器最终能够读取也就是 **Module Instance**。

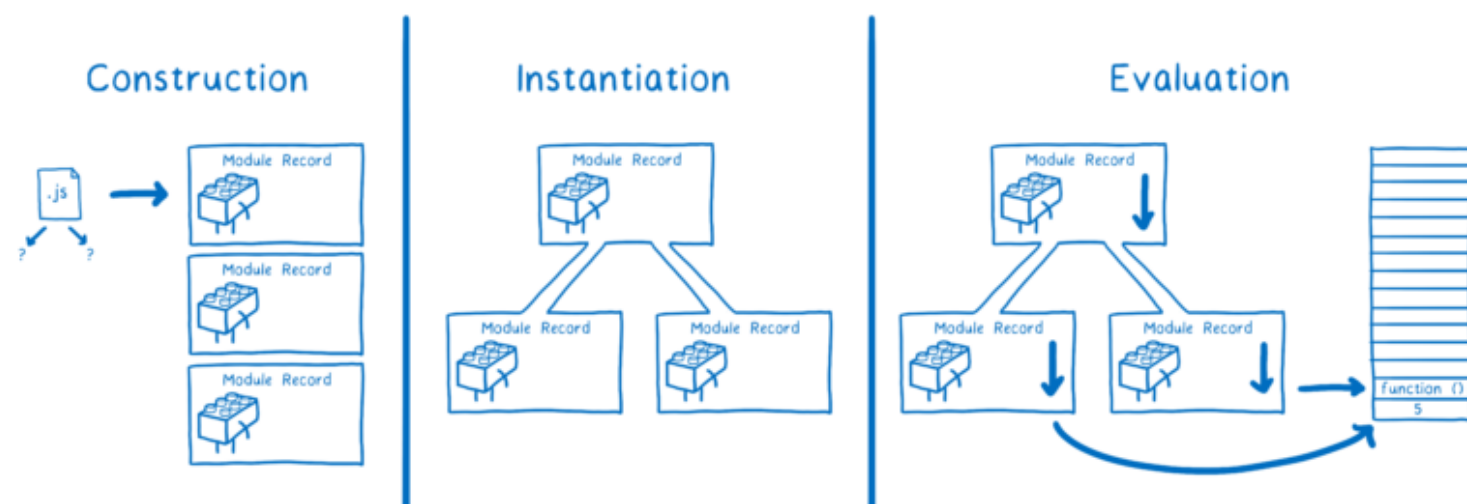
3.2.2 模块加载

模块加载的过程就是从入口文件到拥有一个完整的模块实例图的过程，对于 ES Module 来说，分三步进行：

构造：查找、下载并解析所有文件到模块记录中。

实例化：在内存中寻找一块区域来存储所有导出的变量（但还没有填充值）。然后让 export 和 import 都指向这些内存块。这个过程叫做链接（linking）

求值：在内存块中填入变量的实际值。

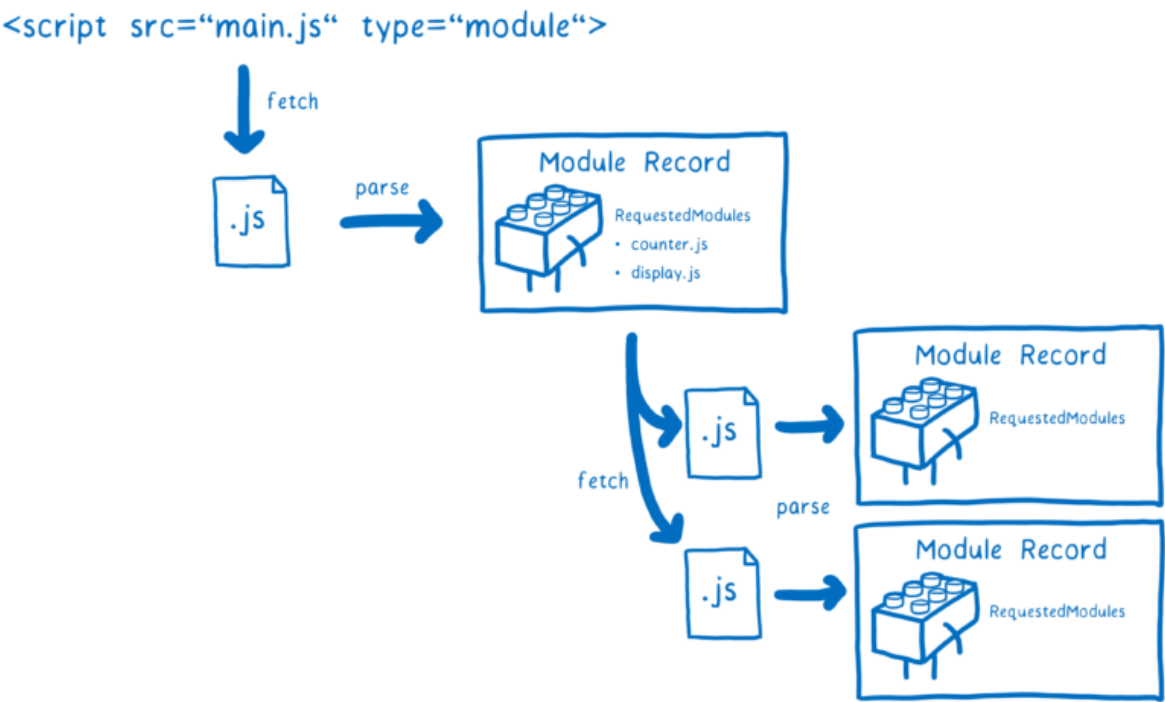


3.2.2.1 构造

在构造阶段，每个模块都会经历三件事情：

- 查找：找出从哪里下载包含该模块的文件（也称为模块解析）；
- 通常会有一个入口文件，然后通过 **import** 代码去寻找与之关联的其他模块，形成一个 **依赖关系树（AST）**；
- 下载：获取文件（从 URL 下载或从文件系统加载）；
- 解析文件前，需要一层一层地遍历树，找出它的依赖项，然后找到并加载这些依赖项；
- 解析：将文件解析为模块记录；
- 把解析出来的模块构成表 称为 **Module Record**（模块记录）。
- **Module Record** 包含了当前模块的 **AST**，引用了哪些模块的变量，以及一些特定属性和方法。
- 一旦 **Module Record** 被创建，它会被记录在模块映射 **Module Map** 中。被记录后，如果再有对相同 **URL** 的请求，**Loader** 将直接采用 **Module Map** 中 **URL** 对应的 **Module Record**。

在构造过程结束时，从主入口文件变成了一堆模块记录 **Module Record**：



3.2.2.2 实例化

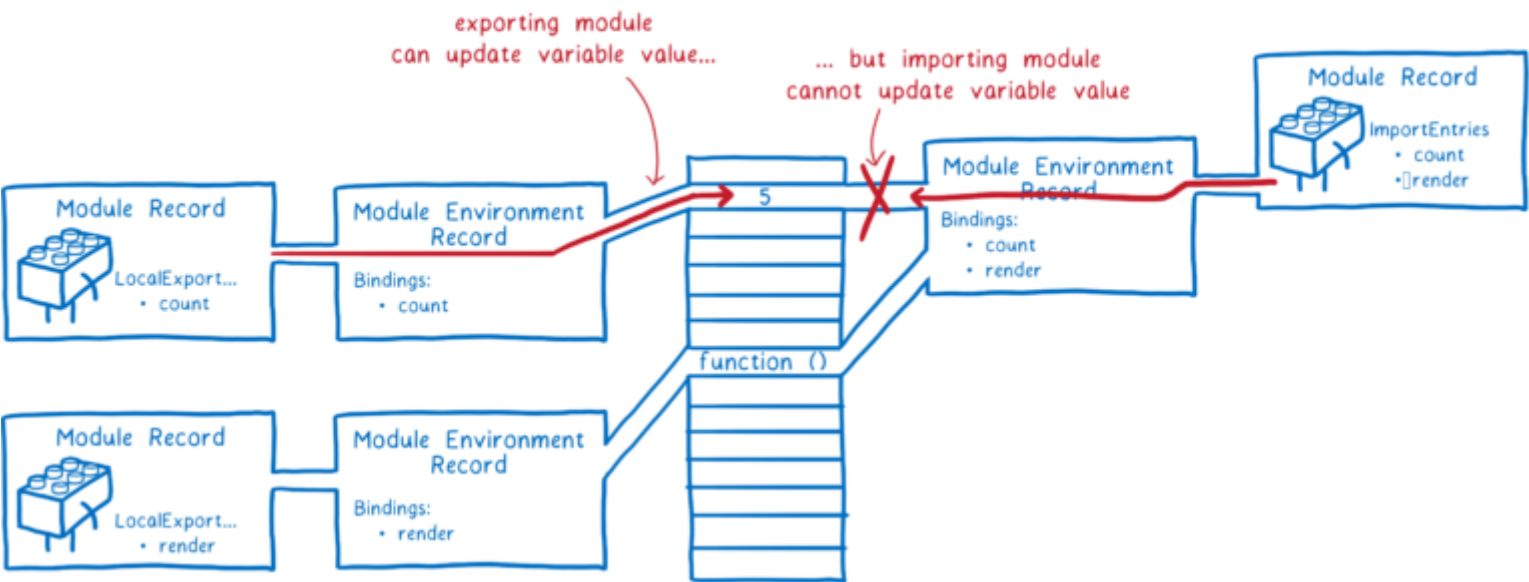
实例化阶段：将构造的模块实例化并将所有实例链接在一起。

模块实例包含两部分：**代码、状态**。

状态存在于内存中，因此实例化步骤就是写入内存。

首先，JS引擎创建一个模块环境记录(`Module Enviroment Record`)来管理 `Module Record` 的变量。然后它在内存中找到所有导出内容对应的位置。模块环境记录将跟踪内存中导出内容对应的位置与导出内容间的联系。

此时内存中的这些位置中还不会存放值，只有在计算后才会有值。

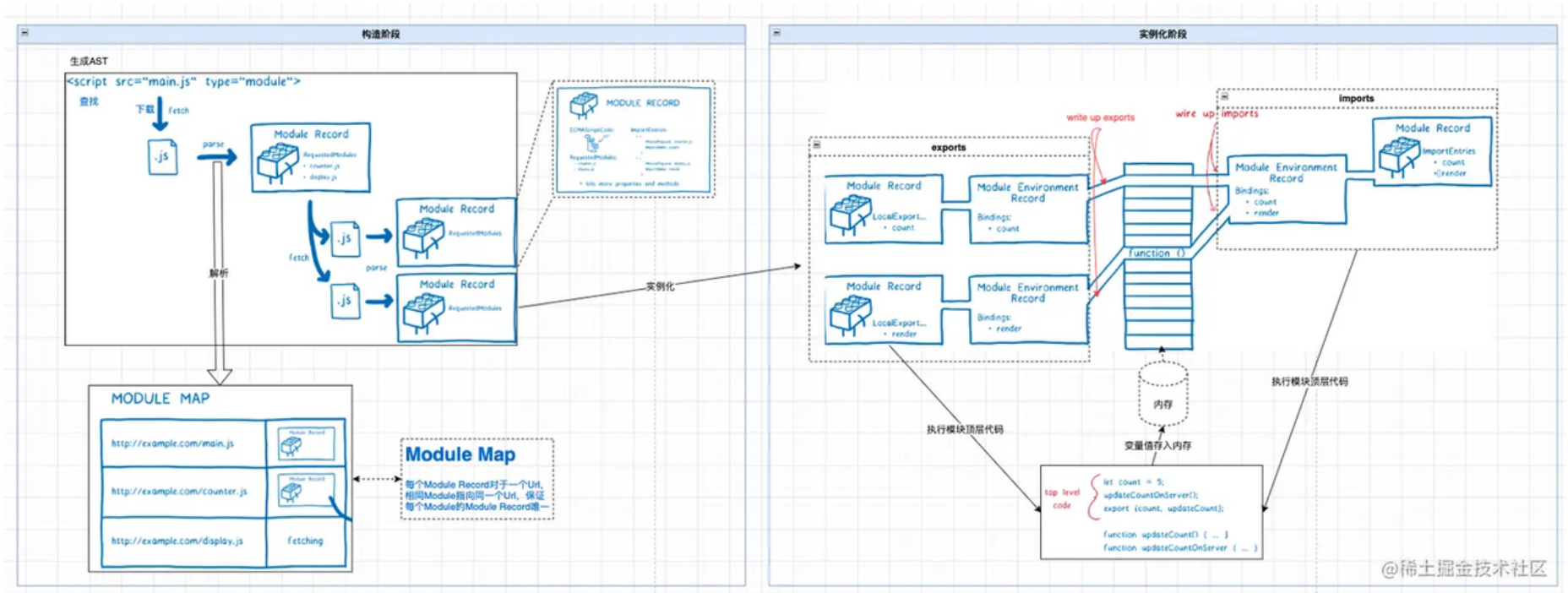


3.2.2.3 求值

最后一步，在内存区中填充绑定的数据的值。

JS 引擎通过执行顶层代码（函数之外的代码，此处可以理解为模块文件中顶层作用域中的代码）来给内存区的引用赋值。

3.2.3 总结



3.3 二者区别

1. ES Module 输出的是值的引用，而 CommonJS 输出的是值的拷贝，一旦输出一个值，模块内部的变化就影响不到这个值；
2. ES Module 是编译时执行，而 CommonJS 模块是在运行时加载；
3. ES6 Module 可以导出多个值，而 CommonJs 是单个值导出；
4. ES6 Module 静态语法只能写在顶层，而 CommonJs 是动态语法可以写在判断里；
5. ES6 Module 的 `this` 是 `undefined`，而 CommonJs 的 `this` 是当前模块；

Code block

```

1  // esm
2  // lib.js
3  export let counter = 3;
4  export function incCounter() {
5    counter++;
6  }
7  // main.js
8  import { counter, incCounter } from './lib';
9  console.log(counter); // 3
10 incCounter();
11 console.log(counter); // 4

```

4. 完整的前端技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



5. Webpack 完整流程

1. 初始化阶段 - `webpack`

- 合并配置项
- 创建 compiler
- 注册插件

2. 编译阶段 - `build`

- 读取入口文件
- 从入口文件开始进行编译
- 调用 loader 对源代码进行转换
- 借助 babel 解析为 AST 收集依赖模块
- 递归对依赖模块进行编译操作

3. 生成阶段 - `seal`

- 创建 chunk 对象
- 生成 assets 对象

4. 写入阶段 - `emit`

5.1 初始化阶段

初始化阶段的逻辑集中在调用 `webpack(config)` 时候，下面我们来看看 `webpack()` 函数体内做了哪些事项。

5.1.1 读取与合并配置信息

通常，在我们的工程的根目录下，会有一个 `webpack.config.js` 作为 `webpack` 的配置来源；

除此之外，还有一种是通过 `webpak bin cli` 命令进行打包时，命令行上携带的参数也会作为 `webpack` 的配置。

在配置文件中包含了我们要让 `webpack` 打包处理的入口模块、输出位置、以及各种 loader、plugin 等；

在命令行上也同样可以指定相关的配置，且 `权重高于` 配置文件。（下面将模拟 `webpack cli` 参数合并处理）

所以，我们在 `webpack` 入口文件这里将先做一件事情：合并配置文件与命令行的配置。

Code block

```
1  // lib/webpack.js
2  function webpack(options) {
3    // 1、合并配置项
4    const mergeOptions = _mergeOptions(options);
5    ...
6  }
7
8  function _mergeOptions(options) {
9    const shellOptions = process.argv.slice(2).reduce((option, argv) => {
10      // argv -> --mode=production
11      const [key, value] = argv.split('=');
12      if (key && value) {
13        const parseKey = key.slice(2);
14        option[parseKey] = value;
15      }
16      return option;
17    }, {});
18    return { ...options, ...shellOptions };
19  }
20
21  module.exports = webpack;
22
```

5.1.2 创建编译器（compiler）对象

好的程序结构离不开一个实例对象，webpack 同样也不甘示弱，其编译运转是由一个叫做 `compiler` 的实例对象来驱动运转。

在 `compiler` 实例对象上会记录我们传入的配置参数，以及一些串联插件进行工作的 `hooks` API。

同时，还提供了 `run` 方法启动打包构建，`emitAssets` 对打包产物进行输出磁盘写入。这部分内容后面介绍。

Code block

```
1  // lib/webpack.js
2  const Compiler = require('./compiler');
3
4  function webpack(options) {
5    // 1、合并配置项
6    const mergeOptions = _mergeOptions(options);
7    // 2、创建 compiler
8    const compiler = new Compiler(mergeOptions);
9    ...
10   return compiler;
11 }
12
13 module.exports = webpack;
```

`Compiler` 构造函数基础结构如下：

Code block

```
1  // core/compiler.js
2  const fs = require('fs');
3  const path = require('path');
4  const { SyncHook } = require('tapable'); // 串联 compiler 打包流程的订阅与通知钩子
5  const Compilation = require('./compilation'); // 编译构造函数
6
7  class Compiler {
8    constructor(options) {
9      this.options = options;
10     this.context = this.options.context || process.cwd().replace(/\\/g, '/');
11     this.hooks = {
12       // 开始编译时的钩子
13       run: new SyncHook(),
14       // 模块解析完成，在向磁盘写入输出文件时执行
15       emit: new SyncHook(),
16       // 在输出文件写入完成后执行
17       done: new SyncHook(),
18     };
19   }
20
21   run(callback) {
22     ...
23   }
24
25   emitAssets(compilation, callback) {
26     ...
27   }
28 }
29
30 module.exports = Compiler;
```

当需要进行编译时，调用 `compiler.run` 方法即可：

Code block

```
1 compiler.run((err, stats) => { ... });
```

5.1.3 插件注册

有 `compiler` 实例对象后，就可以注册配置文件中的一个插件，在合适的时机来干预打包构建。

插件需要接收 `compiler` 对象作为参数，以此来对打包过程及产物产生 `side effect`。

插件的格式可以是函数或对象，如果为对象，需要自定义提供一个 `apply` 方法。常见的插件结构如下：

Code block

```
1 class WebpackPlugin {
2   apply(compiler) {
3     ...
4   }
5 }
```

注册插件逻辑如下：

Code block

```
1 // lib/webpack.js
2 function webpack(options) {
3   // 1、合并配置项
4   const mergeOptions = _mergeOptions(options);
5   // 2、创建 compiler
6   const compiler = new Compiler(mergeOptions);
7   // 3、注册插件，让插件去影响打包结果
8   if (Array.isArray(options.plugins)) {
9     for (const plugin of options.plugins) {
10       if (typeof plugin === "function") {
11         plugin.call(compiler, compiler); // 当插件为函数时
12       } else {
13         plugin.apply(compiler); // 如果插件是一个对象，需要提供 apply 方法。
14       }
15     }
16   }
17   return compiler;
18 }
```

到这里，webpack 的初始工作已经完成，接下来是调用 `compiler.run()` 进入编译构建阶段。

5.2 编译阶段

编译工作的起点是在 `compiler.run`，它会：

- 1. 发起构建通知，触发 `hooks.run` 通知相关插件；
- 2. 创建 `compilation` 编译对象；
- 3. 读取 entry 入口文件；
- 4. 编译 entry 入口文件；

5.2.1 创建 compilation 编译对象

Code block

```

1  // lib/compiler.js
2  class Compiler {
3    ...
4    run(callback) {
5      // 触发 run hook
6      this.hooks.run.call();
7      // 创建 compilation 编译对象
8      const compilation = new Compilation(this);
9      ...
10   }
11 }

```

`compilation` 实例上记录了构建过程中的 `entries`、`module`、`chunks`、`assets` 等编译信息，同时提供 `build` 和 `seal` 方法进行代码构建和代码生成。

Code block

```

1  // lib/compilation.js
2  const fs = require('fs');
3  const path = require('path');
4  const parser = require('@babel/parser');
5  const traverse = require('@babel/traverse').default;
6  const generator = require('@babel/generator').default;
7  const t = require('@babel/types');
8  const { tryExtensions, getSourceCode } = require('./utils');
9
10 class Compilation {
11   constructor(compiler) {
12     this.compiler = compiler;
13     this.context = compiler.context;
14     this.options = compiler.options;
15     // 记录当前 module code
16     this.moduleCode = null;
17     // 保存所有依赖模块对象
18     this.modules = new Set();
19     // 保存所有入口模块对象
20     this.entries = new Map();
21     // 所有的代码块对象
22     this.chunks = new Set();
23     // 存放本次产出的文件对象（与 chunks 一一对应）
24     this.assets = {};
25   }
26   build() {}
27   seal() {}
28 }

```

有了 `compilation` 对象后，通过执行 `compilation.build` 开始模块构建。

Code block

```

1  // lib/compiler.js
2  class Compiler {
3    ...
4    run(callback) {
5      // 触发 run hook
6      this.hooks.run.call();
7      // 创建 compilation 编译对象
8      const compilation = new Compilation(this);
9      // 编译模块
10     compilation.build();

```

```
11    }
12  }
```

5.2.2 读取 entry 入口文件

构建模块首先从 entry 入口模块开始，此时首要工作是根据配置文件拿到入口模块信息。

entry 配置的方式多样化，如：可以不传（有默认值）、可以传入 string，也可以传入对象指定多个入口。

所以读取入口文件需要考虑并兼容这几种灵活配置方式。

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    build() {
5      // 1、读取配置入口
6      const entry = this.getEntry();
7      ...
8    }
9
10   getEntry() {
11     let entry = Object.create(null);
12     const { entry: optionsEntry } = this.options;
13     if (!optionsEntry) {
14       entry['main'] = 'src/index.js'; // 默认找寻 src 目录进行打包
15     } else if (typeof optionsEntry === 'string') {
16       entry['main'] = optionsEntry;
17     } else {
18       entry = optionsEntry; // 视为对象，比如多入口配置
19     }
20     // 相对于项目启动根目录计算出相对路径
21     Object.keys(entry).forEach((key) => {
22       entry[key] = './' + path.posix.relative(this.context, entry[key]);
23     });
24     return entry;
25   }
26 }
```

5.2.3 编译 entry 入口文件

拿到入口文件后，依次对每个入口进行构建。

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    build() {
5      // 1、读取配置入口
6      const entry = this.getEntry();
7      // 2、构建入口模块
8      Object.keys(entry).forEach((entryName) => {
9        const entryPath = entry[entryName];
10       const entryData = this.buildModule(entryName, entryPath);
11       this.entries.set(entryName, entryData);
12     });
13   }
14 }
15
```

构建阶段执行如下操作：

1. 通过 `fs` 模块读取 entry 入口文件内容；
2. 调用 `loader` 来转换（更改）文件内容；
3. 为模块创建 `module` 对象，通过 AST 解析源代码收集依赖模块，并改写依赖模块的路径；
4. 如果存在依赖模块，递归进行上述三步操作；

读取文件内容：

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    buildModule(moduleName, modulePath) {
5      // 1. 读取文件原始代码
6      const originSourceCode = fs.readFileSync(modulePath, 'utf-8');
7      this.moduleCode = originSourceCode;
8      ...
9    }
10 }
```

调用 loader 转换源代码：

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    buildModule(moduleName, modulePath) {
5      // 1. 读取文件原始代码
6      const originSourceCode = fs.readFileSync(modulePath, 'utf-8');
7      this.moduleCode = originSourceCode;
8      // 2. 调用 loader 进行处理
9      this.runLoaders(modulePath);
10     ...
11   }
12 }
```

`loader` 本身是一个 JS 函数，接收模块文件的源代码作为参数，经过加工改造后返回新的代码。

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    runLoaders(modulePath) {
5      const matchLoaders = [];
6      // 1. 找到与模块相匹配的 loader
7      const rules = this.options.module.rules;
8      rules.forEach((loader) => {
9        const testRule = loader.test;
10       if (testRule.test(modulePath)) {
11         // 如: { test: /\.js$/g, use: ['babel-loader'] }, { test: /\.js$/, loader: 'babel-loader' }
12         loader.loader ? matchLoaders.push(loader.loader) : matchLoaders.push(...loader.use);
13       }
14     });
15     // 2. 倒序执行 loader
16     for (let i = matchLoaders.length - 1; i >= 0; i--) {
```

```

17     const loaderFn = require(matchLoaders[i]);
18     // 调用 loader 处理源代码
19     this.moduleCode = loaderFn(this.moduleCode);
20 }
21 }
22 }

```

执行 webpack 模块编译逻辑：

Code block

```

1  // lib/compilation.js
2  class Compilation {
3      ...
4      buildModule(moduleName, modulePath) {
5          // 1. 读取文件原始代码
6          const originSourceCode = fs.readFileSync(modulePath, 'utf-8');
7          this.moduleCode = originSourceCode;
8          // 2. 调用 loader 进行处理
9          this.runLoaders(modulePath);
10         // 3. 调用 webpack 进行模块编译 为模块创建 module 对象
11         const module = this.handleWebpackCompiler(moduleName, modulePath);
12         return module; // 返回模块
13     }
14 }

```

1. 创建 `module` 对象；
2. 对 module code 解析为 `AST` 语法树；
3. 遍历 AST 去识别 `require` 模块语法，将模块收集在 `module.dependencies` 之中，并改写 `require` 语法为 `webpack_require`；
4. 将修改后的 AST 转换为源代码；
5. 若存在依赖模块，深度递归构建依赖模块。

Code block

```

1  // lib/compilation.js
2  class Compilation {
3      ...
4      handleWebpackCompiler(moduleName, modulePath) {
5          // 1. 创建 module
6          const moduleId = './' + path.posix.relative(this.context, modulePath);
7          const module = {
8              id: moduleId, // 将当前模块相对于项目启动根目录计算出相对路径 作为模块ID
9              dependencies: new Set(), // 存储该模块所依赖的子模块
10             entryPoint: [moduleName], // 该模块所属的入口文件
11         };
12
13         // 2. 对模块内容解析为 AST，收集依赖模块，并改写模块导入语法为 __webpack_require__
14         const ast = parser.parse(this.moduleCode, {
15             sourceType: 'module',
16         });
17
18         // 遍历 ast，识别 require 语法
19         traverse(ast, {
20             CallExpression: (nodePath) => {
21                 const node = nodePath.node;
22                 if (node.callee.name === 'require') {
23                     const requirePath = node.arguments[0].value;

```

```

24      // 寻找模块绝对路径
25      const moduleDirName = path.posix.dirname(modulePath);
26      const absolutePath = tryExtensions(
27        path.posix.join(moduleDirName, requirePath),
28        this.options.resolve.extensions,
29        requirePath,
30        moduleDirName
31      );
32      // 创建 moduleId
33      const moduleId = './' + path.posix.relative(this.context, absolutePath);
34      // 将 require 变成 __webpack_require__ 语句
35      node.callee = t.identifier('__webpack_require__');
36      // 修改模块路径 (参考 this.context 的相对路径)
37      node.arguments = [t.stringLiteral(moduleId)];
38
39      if (!Array.from(this.modules).find(module => module.id === moduleId)) {
40        // 在模块的依赖集中记录子依赖
41        module.dependencies.add(moduleId);
42      } else {
43        // 已经存在模块集中。虽然不添加进入模块编译 但是仍要在这个模块上记录被依赖的入口模块
44        this.modules.forEach((module) => {
45          if (module.id === moduleId) {
46            module.entryPoint.push(moduleName);
47          }
48        });
49      }
50    },
51  });
52  });
53
54  // 3、将 ast 生成新代码
55  const { code } = generator(ast);
56  module._source = code;
57
58  // 4、深度递归构建依赖模块
59  module.dependencies.forEach((dependency) => {
60    const depModule = this.buildModule(moduleName, dependency);
61    // 将编译后的任何依赖模块对象加入到 modules 对象中去
62    this.modules.add(depModule);
63  });
64
65  return module;
66 }
67 }

```

通常我们 require 一个模块文件时习惯不去指定文件后缀，默认会查找 .js 文件。

这跟我们在配置文件中指定的 `resolve.extensions` 配置有关，在 `tryExtensions` 方法中会尝试为每个未填写后缀的 Path 应用 `resolve.extensions`：

Code block

```

1  // lib/utils.js
2  const fs = require('fs');
3
4  function tryExtensions(
5    modulePath, extensions, originModulePath, moduleContext
6  ) {
7    // 优先尝试不需要扩展名选项 (用户如果已经传入了后缀, 那就使用用户填入的, 无需再应用 extensions)
8    extensions.unshift('');
9    for (let extension of extensions) {

```

```

10     if (fs.existsSync(modulePath + extension)) {
11         return modulePath + extension;
12     }
13 }
14 // 未匹配对应文件
15 throw new Error(
16     `No module, Error: Can't resolve ${originModulePath} in ${moduleContext}`
17 );
18 }
19
20 module.exports = {
21     tryExtensions,
22     ...
23 }

```

至此，「编译阶段」到此结束，接下来是「生成阶段」 `seal`。

5.3 生成阶段

在「编译阶段」会将一个个文件构建成 `module` 存储在 `this.modules` 之中。

在「生成阶段」，会根据 `entry` 创建对应 `chunk` 并从 `this.modules` 中查找被 `entry` 所依赖的 `module` 集合。

最后，结合 `runtime` webpack 模块机制运行代码，经过拼接生成最终的 `assets` 产物。

Code block

```

1  // lib/compiler.js
2  class Compiler {
3      ...
4      run(callback) {
5          // 触发 run hook
6          this.hooks.run.call();
7          // 创建 compilation 编译对象
8          const compilation = new Compilation(this);
9          // 编译模块
10         compilation.build();
11         // 生成产物
12         compilation.seal();
13         ...
14     }
15 }

```

`entry + module` --> `chunk` --> `assets` 过程如下：

Code block

```

1  // lib/compilation.js
2  class Compilation {
3      ...
4      seal() {
5          // 1、根据 entry 创建 chunk
6          this.entries.forEach((entryData, entryName) => {
7              // 根据当前入口文件和模块的相互依赖关系，组装成为一个个包含当前入口所有依赖模块的 chunk
8              this.createChunk(entryName, entryData);
9          });
10         // 2、根据 chunk 创建 assets
11         this.createAssets();
12     }
13
14     // 根据入口文件和依赖模块组装chunks

```

```

15   createChunk(entryName, entryData) {
16     const chunk = {
17       // 每一个入口文件作为一个 chunk
18       name: entryName,
19       // entry build 后的数据信息
20       entryModule: entryData,
21       // entry 的所依赖模块
22       modules: Array.from(this.modules).filter((i) =>
23         i.entryPoint.includes(entryName)
24       ),
25     };
26     // add chunk
27     this.chunks.add(chunk);
28   }
29
30   createAssets() {
31     const output = this.options.output;
32     // 根据 chunks 生成 assets
33     this.chunks.forEach((chunk) => {
34       const parseFileName = output.filename.replace('[name]', chunk.name);
35       // 为每一个 chunk 文件代码拼接 runtime 运行时语法
36       this.assets[parseFileName] = getSourceCode(chunk);
37     });
38   }
39 }

```

`getSourceCode` 是将 `entry` 和 `modules` 组合而成的 `chunk`，接入到 `runtime` 代码模板之中。

Code block

```

1  // lib/utils.js
2  function getSourceCode(chunk) {
3    const { entryModule, modules } = chunk;
4    return `(() => { var __webpack_modules__ = {      ${modules      }.map((module) => {
return `      '${module.id}': (module) => {      ${module._source}
5      }      `;      })      .join(',')})
6    }; var __webpack_module_cache__ = {}; function __webpack_require__(moduleId) { var
cachedModule = __webpack_module_cache__[moduleId]; if (cachedModule !== undefined) { return
cachedModule.exports; } var module = (__webpack_module_cache__[moduleId] = { exports:
{}, }); __webpack_modules__[moduleId](module, module.exports, __webpack_require__);
return module.exports; } (() => {      ${entryModule._source}
7      })(); })(); `;
8  }
9
10

```

到这里，「生成阶段」处理完成，这也意味着 `compilation` 编译工作的完成，接下来我们回到 `compiler` 进行最后的「产物输出」。

5.4 写入阶段

「写入阶段」比较容易理解，`assets` 上已经拥有了最终打包后的代码内容，最后要做的就是将代码内容写入到本地磁盘之中。

Code block

```

1  // lib/compiler.js
2  class Compiler {
3    ...
4    run(callback) {
5      // 触发 run hook

```

```

6      this.hooks.run.call();
7      // 创建 compilation 编译对象
8      const compilation = new Compilation(this);
9      // 编译模块
10     compilation.build();
11     // 生成产物
12     compilation.seal();
13     // 输出产物
14     this.emitAssets(compilation, callback);
15 }
16
17 emitAssets(compilation, callback) {
18     const { entries, modules, chunks, assets } = compilation;
19     const output = this.options.output;
20
21     // 调用 Plugin emit 钩子
22     this.hooks.emit.call();
23
24     // 若 output.path 不存在, 进行创建
25     if (!fs.existsSync(output.path)) {
26         fs.mkdirSync(output.path);
27     }
28
29     // 将 assets 中的内容写入文件系统中
30     Object.keys(assets).forEach((fileName) => {
31         const filePath = path.join(output.path, fileName);
32         fs.writeFileSync(filePath, assets[fileName]);
33     });
34
35     // 结束之后触发钩子
36     this.hooks.done.call();
37
38     callback(null, {
39         toJSON: () => {
40             return {
41                 entries,
42                 modules,
43                 chunks,
44                 assets,
45             };
46         },
47     });
48 }
49 }

```

至此，webpack 的打包流程就已完成。

5.5 loader

在 `webpack.config.js` 中我们为 `.js` 文件类型配置了一个自定义 loader 来转换文件内容：

Code block

```

1  // webpack.config.js
2  module: {
3      rules: [
4          {
5              test: /\.js/,
6              use: [
7                  path.resolve(__dirname, './loaders/transformArrowFnLoader.js'),
8              ],

```

```
9     },
10    ],
11   },
```

loader 本身是一个函数，接收文件模块内容作为参数，经过改造处理返回新的文件内容。

下面我们在 loaders/transformArrowFnLoader.js 中，对文件中使用到的箭头函数，转换为普通函数，来理解 webpack loader 的作用。

Code block

```
1  // loaders/transformArrowFnLoader.js
2  const parser = require('@babel/parser');
3  const traverse = require('@babel/traverse').default;
4  const generator = require('@babel/generator').default;
5  const t = require('@babel/types');
6
7  function transformArrowLoader(sourceCode) {
8    const ast = parser.parse(sourceCode, {
9      sourceType: 'module'
10   });
11   traverse(ast, {
12     ArrowFunctionExpression(path, state) {
13       const node = path.node;
14       const body = path.get('body');
15       const bodyNode = body.node;
16       if (bodyNode.type !== 'BlockStatement') {
17         const statements = [];
18         statements.push(t.returnStatement(bodyNode));
19         node.body = t.blockStatement(statements);
20       }
21       node.type = "FunctionExpression";
22     }
23   });
24   const { code } = generator(ast);
25
26   return code;
27 }
28
29 module.exports = transformArrowLoader;
30
31
```

5.6 Plugin

从上面介绍我们了解到，每个插件都需要提供一个 apply 方法，此方法接收 compiler 作为参数。

通过 compiler 可以去订阅 webpack 工作期间不同阶段的 hooks，以此来影响打包结果或者做一些定制操作。

下面我们编写自定义插件，绑定两个不同时的 compiler.hooks 来扩展 webpack 打包功能：

- hooks.emit.tap 绑定一个函数，在 webpack 编译资源完成，输出写入磁盘前执行（可以做清除 output.path 目录操作）；
- hooks.done.tap 绑定一个函数，在 webpack 写入磁盘完成之后执行（可以做一些静态资源 copy 操作）。

Code block

```
1  // plugins/custom-webpack-plugins
2  const fs = require('fs-extra');
3  const path = require('path');
4
```

```
5  class CustomWebpackPlugin {
6    apply(compiler) {
7      const outputPath = compiler.options.output.path;
8      const hooks = compiler.hooks;
9
10     // 清除 build 目录
11     hooks.emit.tap('custom-webpack-plugin', (compilation) => {
12       fs.removeSync(outputPath);
13     });
14
15     // copy 静态资源
16     const otherFilesPath = path.resolve(__dirname, '../src/otherfiles');
17     hooks.done.tap('custom-webpack-plugin', (compilation) => {
18       fs.copySync(otherFilesPath, path.resolve(outputPath, 'otherfiles'));
19     });
20   }
21 }
22
23 module.exports = CustomWebpackPlugin;
```